Discrete Structures for Computer Science

William Garrison
bill@cs.pitt.edu
6311 Sennott Square

Lecture #5: Logic Programming and Nested Quantifiers



University of Pittsburgh

Based on materials developed by Dr. Adam Lee



Today's topics

- Applications of predicate logic
- Nested quantifiers

Logic programming enables automated reasoning



Prolog

- Programming in logic
- Developed in the 1970s for AI purposes

Datalog

- Logical formalization of databases
- Developed in the 1980s

For our purposes, we can consider Prolog and Datalog to be the same, though in reality they have very important differences.

Two main constructs:

- Facts
  - ✧ instructor(bill, cs441)
  - ✧ student(smith, cs441)
- Rules
  - ✧ teaches(P,S) :- instructor(P,C), student(S,C)

Lower case = constant

Upper case = variable



Rules and facts define predicates

Facts define predicates by explicitly listing elements that satisfy those predicates

- "Dr. Garrison is the instructor for CS441"
  ≡ instructor(bill, cs441)

Rules define predicates by combining previously specified predicates

- "Professors teach the students enrolled in the courses for which they are the instructor" ≡
  teaches(P,S) :- instructor(P,C), student(S,C)

Prolog is an environment that lets us issue queries to determine which predicates are true!

A Security Example

```
grant(U, projector) :- located(U, 105), role(U, presenter)
located(U, R) :- owns(U, D), dev_loc(D, R)

role(bob, presenter)      owns(alice, laptop12)    dev_loc(laptop12, 105)
role(carol, presenter)    owns(bob, tablet23)      dev_loc(tablet23, 105)
                          owns(carol, cell42)      dev_loc(cell42, 105)
```

Knowledge base

Can Bob run the projector?

- *Query:* ?grant(bob, projector)
- *Solution:* true

Who is in room 105?

- *Query:* ?located(X, 105)
- *Solution:* alice, bob, carol

Write and evaluate the following queries



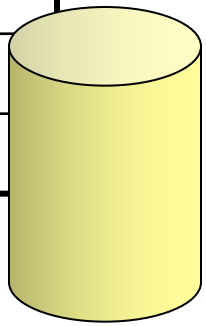
```
grant(U, projector) :- located(U, 105), role(U, presenter)
located(U, R) :- owns(U, D), dev_loc(D, R)

role(bob, presenter)      owns(alice, laptop12)    dev_loc(laptop12, 105)
role(carol, presenter)    owns(bob, tablet23)      dev_loc(tablet23, 105)
                          owns(carol, cell42)      dev_loc(cell42, 105)
```

- Can Alice use the projector?
  - ?grant(alice, projector)
  - false
- Can Carol use the projector
  - ?grant(carol, projector)
  - true
- Which devices does Alice own?
  - ?owns(alice, X)
  - laptop12

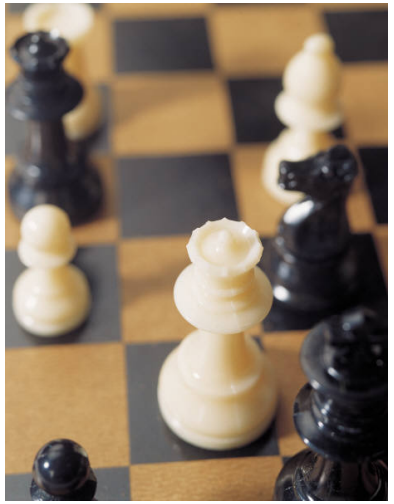
Logic programming is a useful tool!

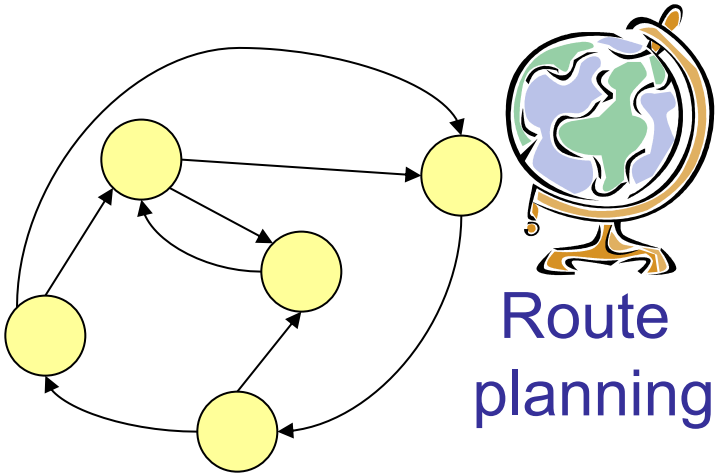
Name	Age	Phone
Alice	19	555-1234
Danielle	33	555-5353
Zach	27	555-3217
Charlie	21	555-2335



Databases

Artificial Intelligence





Route planning



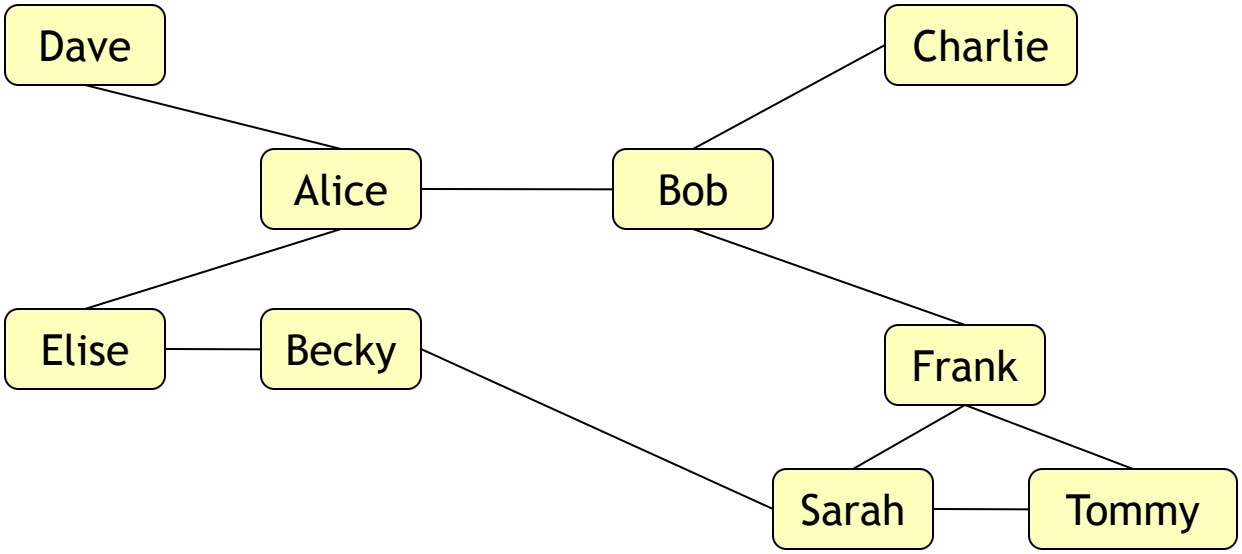
Security

Just for grins…

If you are interested in playing around with logic programming, download SWI-Prolog

- URL: http://www.swi-prolog.org/

This (free) package is a runtime environment in which you can write logic programs and evaluate queries.





Nested quantifiers!?!?

Many times, we need the ability to nest one quantifier within the scope of another quantifier

Example: All integers have an additive inverse. That is, for any integer x, we can choose an integer y such that the sum of x and y is zero.

There is no way to express this statement using only a single quantifier!

Deciphering nested quantifiers isn't as scary as it looks…

… if you remember to read from left to right!

$$\forall x \; \exists y \; \forall z \; [(x + y) \times z = 0]$$

For all x…

… there exists a y such that…

… for all z…

… (x + y) × z = 0

And think about scope of variables like with programming!

A few more examples…



This is the commutative law for addition!

$\forall x\ \forall y\ (x + y = y + x)$

- For all integers x and for all integers y, x + y = y + x

This is the associative law for addition!

$\forall x\ \forall y\ \forall z\ [(x+y)+z = x+(y+z)]$

- For all integers x, for all integers y, and for all integers z, (x+y)+z = x+(y+z)

$\exists x\ \forall y\ (x \times y = 0)$

- There exists an x such that for all y, $x \times y = 0$

Since we always read from left to right, the order of quantifiers matters!

Consider: $\forall x \, \exists y \, (x + y = 0)$

Clearly true!
Just set y = -x

Transpose: $\exists y \, \forall x \, (x + y = 0)$

Not true…

Remember: As long as you read from left to right, you won't have any problems!

Many mathematical statements can be translated into logical statements with nested quantifiers



Translating mathematical expressions is often easier than translating English statements!

Steps:

1. Rewrite statement to make quantification and logical operators more explicit
2. Determine the order in which quantifiers should appear
3. Generate logical expression

Let's try a translation…



Universal quantifier

Statement: Every real number except zero has a multiplicative inverse

x × y = 1

Singular—suggestive of an existential quantifier

∀x

Rewrite: For every real number x, if x ≠ 0, then there exists a real number y such that x × y = 1.

… ∃y (x × y = 1)

(x ≠ 0) → …

More examples…

Statement: The product of any two negative integers is always positive

Statement: For any real number a, it is possible to choose real numbers b and c such that $a^2 + b^2 = c^2$

Translating quantified statements to English is as easy as reading a sentence!



Let:

- C(x) ≡ x is enrolled in CS441
- M(x) ≡ x has an MP3 player
- F(x, y) ≡ x and y are friends
- Domain of x and y is "all students"

Statement: $\forall x\,[C(x) \rightarrow M(x) \vee (\exists y\,(F(x,y) \wedge M(y))]$

For every student x…

… if x is enrolled in CS441, then…

… x has an MP3 player…

… or there exists another student y such that…

… x and y are friends…

… and y has an MP3 player.

Translate the following expressions into English



Let:

- O(x,y) ≡ x is older than y
- F(x,y) ≡ x and y are friends
- The domain for variables x and y is “all students”

Statement: $\exists x \, \forall y \, O(x,y)$

Statement: $\exists x \, \exists y \, [F(x,y) \wedge \forall z \, [(y \neq z) \rightarrow \neg F(x,z)]]$



In-class exercises

Problem 1: Translate the following mathematical statement into predicate logic: Every even number is a multiple of 2. Assume that the predicate E(x) means "x is even."

- **Hint:** What does "x is a multiple of 2" mean algebraically? Try not to use "mod."

Problem 2: Translate the following expressions into English. Assume that C(x) means "x has a car", F(x,y) means "x and y are friends", and S(x) means "x is a student."

- $\forall x\ (S(x) \rightarrow C(x) \vee \exists y\ [F(x,y) \wedge C(y)])$
- $\forall x\ \exists y\ \exists z\ [C(x) \vee (F(x,y) \wedge C(y)) \vee (F(x,y) \wedge F(y,z) \wedge C(z))]$

Translating from English to a logical expression with nested quantifiers is a little bit more work…

Steps:

1. If necessary, rewrite the sentence to make quantifiers and logical operations more explicit
2. Create propositional functions to express the concepts in the sentence
3. State the domains of the variables in each propositional function
4. Determine the order of quantifiers
5. Generate logical expression

Let's try an example…

Universal quantifier

Statement: Every student has asked at least one professor a question.

Existential quantifier

Rewrite: For every person x, if x is a student, then there exists a professor whom x has asked a question.

Let:

- S(x) ≡ x is a student
- P(x) ≡ x is a professor
- Q(x,y) ≡ x has asked y a question

Domains for x and y are "all people"

Translation: $\forall x\,(S(x) \rightarrow \exists y\,[P(y) \wedge Q(x,y)])$

Translate the following from English

Statement: There is a man who has tasted every type of beer.

Let:

Domain: all people

Domain: all drinks

Domains: x = all people,
y = all drinks

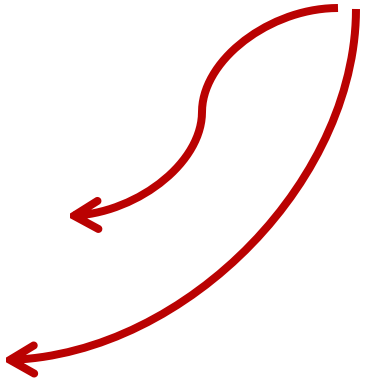
Translation:

Negating expression with nested quantifiers is actually pretty straightforward…

… you just repeatedly apply DeMorgan's laws!

$\neg[\exists x\,(M(x) \wedge \forall y\,[B(y) \rightarrow T(x,y)])]$

$a \rightarrow b \equiv \neg a \vee b$



In English: For all people x, if x is a man, then there exists some type beer that x has not tasted.

Alternatively: No man has tasted every type of beer.

A few stumbling blocks…

Whether the negation sign is on the inside or the outside of a quantified statement makes a big difference!

Example: Let T(x) ≡ "x is tall". Consider the following:

- ¬∀x T(x)
  - "It is not the case that all people are tall."
- ∀x ¬T(x)
  - "For all people x, it is not the case that x is tall."

Note: ¬∀x T(x) = ∃x ¬T(x) ≠ ∀x ¬T(x)

Recall: When we push negation into a quantifier, DeMorgan's law says that we need to switch the quantifier!

A few stumbling blocks…

Let: C(x) ≡ "x is enrolled in CS441"
S(x) ≡ "x is smart."

Question: The following two statements look the same, what's the difference?

- ∃x [C(x) ∧ S(x)]
- ∃x [C(x) → S(x)]

There is a smart student in CS441.

There exists a student x such that if x is in CS441, then x is smart.

Subtle note: The second statement is true if there exists one person not in CS441, because F→F or F→T.

Negate $\forall x\ (S(x) \rightarrow \exists y\ [P(y) \wedge Q(x,y)])$



$\neg\forall x\ (S(x) \rightarrow \exists y\ [P(y) \wedge Q(x,y)])$

In English: There exists a student x such that for all people y, if y is a professor then x has not asked y a question.

Alternatively: There exists a student that has never asked any professor a question.



In-class exercises

Problem 3: Translate the following English sentences into predicate logic.

a) Every student has at least one friend that is dating a Steelers fan.

b) If a person is a parent and a man, then they are the father of some child.

Problem 4: Negate the results from Problem 3 and translate the negated expressions back into English.



Final Thoughts

- Logic programming is an interesting application of predicate logic that is used throughout computer science
- Quantifiers can be nested
  - Nested quantifiers are read left to right
  - Order is important!
  - Translation and negation work the same as they did before!
- Next lecture:
  - Rules of inference
  - Please read sections 1.6–1.7